Pokemon VGC Winner Predictor

Baseline Reproduction - Technical Memo

Analytic Boost - Group 52

1. Repository Submission

- Repository URL: https://github.com/abbas6115/vgc-matchup-predictor
- Release Tag: v0.3.0

2. Reproducible Environment & Repository Setup

The assignments codebase is tagged under "v0.1.0-m3". By running "pip install -r requirements.txt" users can recreate the environment. Running "`pip install -e .`" in the project root directory installs everything in src as its own python package so it can be used on the root directory. The stochastic processes like train/test splits, use a variable called seed which is locked to 42.

3. Working Data Pipeline

src/vgc_matchup_predictor/data/ contains the scripts that download and process the json files.

1. Ingestion and Cleaning: Download_hf.py downloads the files of battle log data from hugging face. process_json.py processes the logs to return json of features of interest from the battle log.

2. Splitting: We split the reg m-a data into an 80/20 train/test split and use reg m-b as a validation set.

3. Preprocessing: Applied MultiLabelBinarizer as features have lists of pokemon that are different each battle (ex.back pokemon). Target label 'Winner' has binary encoding to signify if p1 wins (1), or p2 wins (0).

4. Baseline Methodology

The following methods were chosen to establish a benchmark for future model development

● Naive Benchmark: A Majority Class Classifier. Since p1 has slight win majority (52:48), model predict p1 to win each time

● Course-Level Benchmark: logistic regression of max_iter(1000) to model linear classification. Classifies if the winner is p1 or p2.

5. Baseline Table

Model	Accuracy	Precision	Recall	F1-score
Naive benchmark – majority class	Test: 52.7% Validate: 47.5%	Test: 52.7% Validate: 47.5%	Test: 100% Validate: 100%	Test: 69% Validate: 64%
Naive benchmark – winrate sum	Test: 55.8% Validate: 50%	Test: 58.2% Validate: 47.5%	Test: 57% Validate: 50%	Test: 57.6% Validate: 48.7%
Baseline – Logistic Regression	Test: 56.4% Validate: 49.7%	Test: 58.9% Validate: 47.1%	Test: 57.1% Validate: 48.1%	Test: 58% Validate: 47.6%

6. Discrepancy Analysis

In-distribution evaluation on Reg. M-A shows that Logistic Regression outperforms naive benchmarks across primary metrics–getting 56.41% accuracy and a 58% F1-score compared to 55.8% accuracy for the winrate sum benchmark–by optimizing weights on differential team multi-hot vectors. However, both models show significant performance decay on Reg. M-B validation, dropping to 49.7% accuracy for Logistic Regression and 50% for winrate sum due to new Pokémon, items, etc. being introduced in the latter Regulation causing data drift and Pokémon existing outside the model's vocabulary. Finally, although the majority class baseline produces an inflated F1-score (69% in Reg. M-A) due to a forced 100% recall artifact, accuracy and precision confirm that Logistic Regression establishes the strongest in-distribution performance floor.

AI Use Acknowledgement: ChatGPT was utilized to assist with drafting and determining some key points/values to discuss in discrepancy analysis

7. Start / Stop / Continue Plan

Start: Implement two more features; speed and type matchup comparisons. This gives model information that is significant to human players/tournament play and will likely increase performance statistics. Possible imputation on natures in reg m-a using decision trees.

Stop: Leaving required deliverables uncompleted until before the submission date

Continue: Communicating when new branches are being made, pull requesting, and merging. Keep file paths dynamic using the pathlib module to handle file paths within the project for better reproducibility on different devices.